



node pack

Contents

Click any entry in a PDF viewer to jump directly to that chapter. Page numbers are generated from the finished layout, not estimated from the source text.

The Six Nodes	8	Pass 3	21
The Core Philosophy.....	8	Pass 4	21
Installation	9	Rule format.....	21
Requirements	9		
Install the pack	9		
Included Starter Content	10	Remove my training-version suffix.....	
Included workflow.....	10	Case sensitivity	22
Included prompt logs.....	10	Regex key	23
Included outfit logs	10	Cleanup recipes	23
Where the starter logs go	11	Loader Core Controls.....	24
Quick-Start Wiring	11	diffusionmodel	24
Chapter 1: Loader Core	14	weightdtype.....	24
Legendary Feature: Folder-Scoped LoRA		foldername	24
Cycling	14	mainenabled.....	25
Why parent folders appear	15	mainlora	25
[All LoRA folders]	15	mainstrength	25
[LoRA root only]	15	includesubfolders	25
Selecting a specific folder	16	loopfolder	25
Changing the folder	16	controlaftergenerate.....	25
includesubfolders off	16	skipnoneduringcycle.....	25
includesubfolders on	17	offname	25
The sealed cycling pool	17	autocleanname.....	26
Queue-time progression	18	cleanuprules	26
Progression modes	18	Inactive metadata scrubbing	26
loopfolder	18	Secondary LoRA Stack.....	26
skipnoneduringcycle.....	19	Loader Core Outputs	27
Random mode	19	model	27
foldercount	19	diffusionmodelfile	27
The Three-Name System	19	diffusionmodelstem	28
mainfile.....	20	mainfile.....	28
rawstem.....	20	rawstem.....	28
cleanname.....	20	cleanname.....	28
Name Cleanup and Regex Guide	20	appliedloras	29
Pass 1	21	mainactive.....	29
Pass 2	21	foldercount	29
		Chapter 2: Prompt Core	30

What Is a Prompt Log? 30

Making Your Own Logs 30

Create a log 31

Prompt-log example 31

Outfit-log example 31

Scene-log example 32

The NAME Pipeline 32

Step 1: Loader Core sees the LoRA 32

Step 2: Loader Core creates cleanname 32

Step 3: Connect the nodes 33

Step 4: Write a prompt containing the literal token 33

Step 5: Prompt Core replaces every exact NAME 33

Step 6: Outfit replacement occurs 33

Token matching is literal and case-sensitive 33

Prompt Progression 34

Queue-time behavior 34

Modes 34

Raw index versus resolved index 34

Prompt Core Controls 35

promptsources 35

manualprompt 35

promptlogfile 35

promptmode 36

promptindex 36

promptloop 36

outfitenabled 36

outfitlogfile 36

outfittoken 36

outfitmode 36

outfitindex 36

outfitloop 37

sceneenabled 37

scenelogfile 37

scenetoken 37

scenemode 37

sceneindex 37

sceneloop 37

nametoken 37

namevalue 37

itemtoken 38

itemvalue 38

suffixenabled 38

suffix1 and suffix2 38

suffixseparator 38

skipblanklines 38

cleanupenabled 39

cleanuprules 39

savedprompt 39

Prompt Cleanup 39

What the defaults remove 39

Prompt Core Outputs 40

finalprompt 40

sourceprompt 40

promptline 40

outfitline 40

sceneline 41

promptindexresolved 41

outfitindexresolved 41

sceneindexresolved 41

promptcount 41

outfitcount 41

scenecount 41

promptfile 41

outfitfile 41

sceneindex 41

Chapter 3: Generation Core 42

Why Generation Core Exists 42

Generation Core Controls 42

model 42

positivetext 43

clipname 43

cliptype 43

clipdevice 43

vaename 43

resolutionmode 44

customwidth 44

customheight 44

aspectpreset 44

megapixels 44

batchsize 44

steps 45

cfg 45

samplername 45

scheduler.....	45	quality	57
denoise	45	counterdigits.....	57
shift	45	savepromptjson	57
seed	45	saveworkflowjson	57
Seed System.....	46	savecivitaiparameters	57
Randomize Each Time	46	cleanname.....	57
New Fixed Random	46	rawstem.....	57
Use Last Queued Seed.....	46	modelname	58
Copy Last Used Seed	46	seed.....	58
Generation Core Outputs.....	47	promptindex	58
samples	47	outfitindex	58
vae	47	sceneindex	58
seedused	47	promptfile	58
width.....	47	outfitfile	58
height	47	scenefile	58
generationinfo	47	savedpath.....	58
Chapter 4: Output Core.....	49	generationinfo	58
Why Output Core Exists.....	49	appliedloras	59
Folder and Filename Construction.....	50	Output Core Outputs	59
Subfolder builder	50	images.....	59
Filename builder	50	savepath.....	59
Sanitization.....	50	subfolder	59
Output Variable Key	51	filenameprefix	59
Recommended index connections.....	51	Chapter 5: Preview Core	60
Recommended file connections.....	52	Why Preview Core Exists	60
Naming recipes.....	52	Preview Core Controls	61
Metadata and Civitai Resource Detection	53	images.....	61
Why appliedloras matters.....	53	fitmode	61
Metadata can include	53	backgroundmode.....	62
Hash cache	54	backgroundcolor.....	62
Civitai matching.....	54	Output: images.....	62
Output Core Controls	55	Chapter 6: Persist + Display Resolved	63
images.....	56	Prompt.....	63
outputroot.....	56	Controls	63
subfolderliteral	56	text	63
subfoldervar1 through subfoldervar4	56	savedprompt	63
subfolderdelimiter	56	Output: text.....	63
filenameliteral	56	Complete Wiring Guide	65
filenamevar1 through filenamevar6	56	Essential production wiring	65
filenamedelimiter	56	Optional prompt-log naming wiring.....	66
extension.....	57	Optional resolved-prompt display	66

Workflow Recipes	67
Recipe 1: Test every checkpoint for one identity	67
Recipe 2: One identity, randomized outfits	67
Recipe 3: Pair prompt and outfit lines sequentially.....	68
Recipe 4: Persistent style LoRA plus cycling identities	68
Recipe 5: Human folders, forensic filenames.....	69
Troubleshooting Guide	70
The main LoRA dropdown shows unrelated files	70
The main LoRA changes when I select a folder	70
The primary cycle keeps producing None	70
The primary LoRA is selected but does not affect the model.....	70
cleannname still contains suffixes	71
cleannname became empty	71
NAME remains in the prompt	71
OUTFIT or SCENE remains in the prompt	72
Randomized log indexes become enormous	72
Random mode keeps selecting the final log line	72

New log files do not appear	72
Generation Core does not reproduce the last image	72
Output Core creates an unexpected filename	73
Output Core shows an empty prompt, model, or LoRA in metadata	73
Civitai does not recognize a resource....	73
Preview Core says “Waiting for image...”	74
The saved PNG does not reload the workflow	74
Glossary	75
Active LoRA.....	75
Clean Name	75
Main LoRA.....	75
Raw Stem	75
Resolved Index	75
Raw Index.....	75
Prompt Log	75
Token	75
Queue-Time Progression	75
Applied LoRAs	75
Filename Prefix	75
Compatibility and Release Scope	76
Credits	76
License	76

Control Finder

Looking for one specific field, button, input, or output? This alphabetical index jumps to the explanation and prints its final PDF page number.

[All LoRA folders].....	15	height.....	47
[LoRA root only].....	15	images.....	61
appliedloras	59	includesubfolders	25
aspectpreset	44	itemtoken	38
autocleanname	26	itemvalue	38
backgroundcolor	62	loopfolder	25
backgroundmode.....	62	mainactive	29
batchsize.....	44	mainenabled.....	25
cfg	45	mainfile	28
cleanname.....	57	mainlora	25
cleanupenabled.....	39	mainstrength	25
cleanuprules	39	manualprompt.....	35
clipdevice	43	megapixels	44
clipname	43	model	42
cliptype	43	modelname.....	58
controlaftergenerate	25	nametoken.....	37
counterdigits	57	namevalue.....	37
customheight	44	offname	25
customwidth.....	44	outfitcount.....	41
denoise	45	outfitenabled	36
diffusionmodel.....	24	outfitfile	58
diffusionmodelfile.....	27	outfitindex.....	58
diffusionmodelstem.....	28	outfitindexresolved	41
extension.....	57	outfitline	40
filenamedelimiter	56	outfitlogfile	36
filenameliteral	56	outfitloop	37
filenameprefix.....	59	outfitmode	36
filenamevar1 through filenamevar6.....	56	outfittoken.....	36
finalprompt.....	40	outputroot	56
fitmode	61	positivetext	43
foldercount.....	29	promptcount.....	41
foldername	24	promptfile	58
generationinfo.....	58	promptindex.....	58

promptindexresolved	41	sceneloop	37
promptline	40	scenemode	37
promptlogfile	35	scenetoken	37
promptloop	36	scheduler	45
promptmode	36	seed	58
promptsources	35	seedused	47
quality	57	shift	45
rawstem	57	skipblanklines	38
resolutionmode	44	skipnoneduringcycle	25
samplername	45	sourceprompt	40
samples	47	steps	45
savecivitaiparameters	57	subfolder	59
savedpath	58	subfolderdelimiter	56
savedprompt	63	subfolderliteral	56
savepath	59	subfoldervar1 through subfoldervar4	56
savepromptjson	57	suffix1 and suffix2	38
saveworkflowjson	57	suffixenabled	38
scenecount	41	suffixseparator	38
sceneenabled	37	text	63
sceneindex	58	vae	47
sceneindexresolved	41	vaename	43
sceneline	41	weightdtype	24
scenelogfile	37	width	47

A compact ComfyUI node pack for LoRA testing, prompt-log automation, Krea2 generation, metadata-aware output, and image previews with pizazz.

The Sick Ollie Node Pack is a connected production system built around one guiding idea:

Create information once, transform it intentionally, and carry it forward to every place that needs it.

A LoRA filename can become a clean character name. That name can replace **NAME** inside a prompt. The exact original filename can remain available for traceable output names. The active LoRA stack can become metadata. The resolved prompt can feed generation, remain visible after execution, and survive inside the saved PNG. The randomized seed actually used can return as an output and be written into the final metadata.

This guide explains not only **what** every node does, but **why** each control exists, what every input and output means, and how the six nodes cooperate as one workflow.

The Six Nodes

Node	Role
Loader Core	Loads the diffusion model, cycles a primary LoRA from a chosen folder, applies secondary LoRAs, and creates useful identity strings.
Prompt Core	Builds the final prompt from manual text or reusable logs, then replaces NAME , OUTFIT , SCENE , and ITEM .
Generation Core	Loads the text encoder and VAE, encodes the prompt, creates the latent, samples the image, and reports the exact settings used.
Output Core	Builds folders and filenames, saves the image, embeds workflows, hashes resources, and writes Civitai-readable metadata.
Preview Core	Displays the image with practical fit and background modes while passing the original image through unchanged.
Persist + Display Resolved Prompt	An optional text passthrough that preserves and displays any final resolved prompt inside saved PNG workflows.

The Core Philosophy

A conventional workflow often reconstructs the same information several times:

- one node selects a LoRA
- another text node manually removes the LoRA's suffix

- another node types the character name again
- another save node manually reconstructs the model and LoRA names
- another metadata node guesses what resources were used

The Sick Ollie workflow avoids that duplication.

Loader Core already knows the exact LoRA, so it deliberately produces several versions of its name:

```
main_file = exact selected LoRA path and filename
raw_stem  = exact filename without folder or extension
clean_name = human-friendly name after cleanup rules
```

Each version has a different job:

- **main_file** preserves the exact source
- **raw_stem** preserves the exact trained version
- **clean_name** becomes the human-facing name used in prompts and tidy folders

The system does not discard useful information. It refines it into the form needed by each downstream job.

Prima Tip: Use `clean_name` when you mean “Who or what is this?” Use `raw_stem` when you mean “Which exact file produced this?”

Installation

Requirements

- ComfyUI
- rgthree-comfy

rgthree-comfy is required for Loader Core's enhanced secondary-LoRA interface.

Install the pack

1. Extract the release archive.
2. Locate the folder named `ComfyUI-SickOllie`.
3. Copy it into:

```
ComfyUI/custom_nodes/
```

1. Confirm that rgthree-comfy is installed.
2. Restart ComfyUI completely.
3. Hard-refresh the browser with `Ctrl+F5`.

The final structure should resemble:

```
ComfyUI/  
└─ custom_nodes/  
    └─ ComfyUI-SickOllie/  
        └─ rgthree-comfy/
```

Included Starter Content

The release package includes more than the six custom nodes. It also includes a ready-to-open workflow and a starter library that demonstrates the log system in real use.

Included workflow

```
Starter Content/workflows/SickOllie_NodePack.json
```

Open the JSON through ComfyUI's workflow menu or drag it onto the canvas. The wiring is already assembled. Local diffusion-model, text-encoder, VAE, and LoRA selections may need to be reassigned to files installed on the new machine.

Included prompt logs

Five prompt-log files are included:

```
prompts/clouds_yearbook.txt  
prompts/Insert Outfit Logs/All_Prompts_OUTFIT.txt  
prompts/Insert Outfit Logs/suburban_cinematic_portrait_OUTFIT_prompt_log_100.txt  
prompts/Name Logs/FACEOUT_POSTERS_NAME_ONLY.txt  
prompts/Roadtrip Prompt Logs.txt
```

The starter set deliberately demonstrates different roles:

- ordinary reusable prompt lines
- prompts containing **OUTFIT** for outfit-log insertion
- prompts containing **NAME** for Loader Core name automation and typography experiments
- large cinematic and location-based prompt libraries

Included outfit logs

Six outfit-log files are included:

```
outfits/egirl_catgirl_outfit_log_300.txt  
outfits/fetish_roleplay_privatewear_full_outfit_log.txt  
outfits/innocent_until_worn_like_this_220_prompt_log.txt  
outfits/OUTFITS_FULL_2453.txt  
outfits/OUTFITS_Torso_Only.txt  
outfits/single_item_bare_100_prompt_log.txt
```

These are starter libraries, not required dependencies. Keep, edit, replace, subdivide, or remove them according to the type of work being produced.

Where the starter logs go

The package mirrors the destination path:

```
Starter Content/input/SickOllieLogs/
```

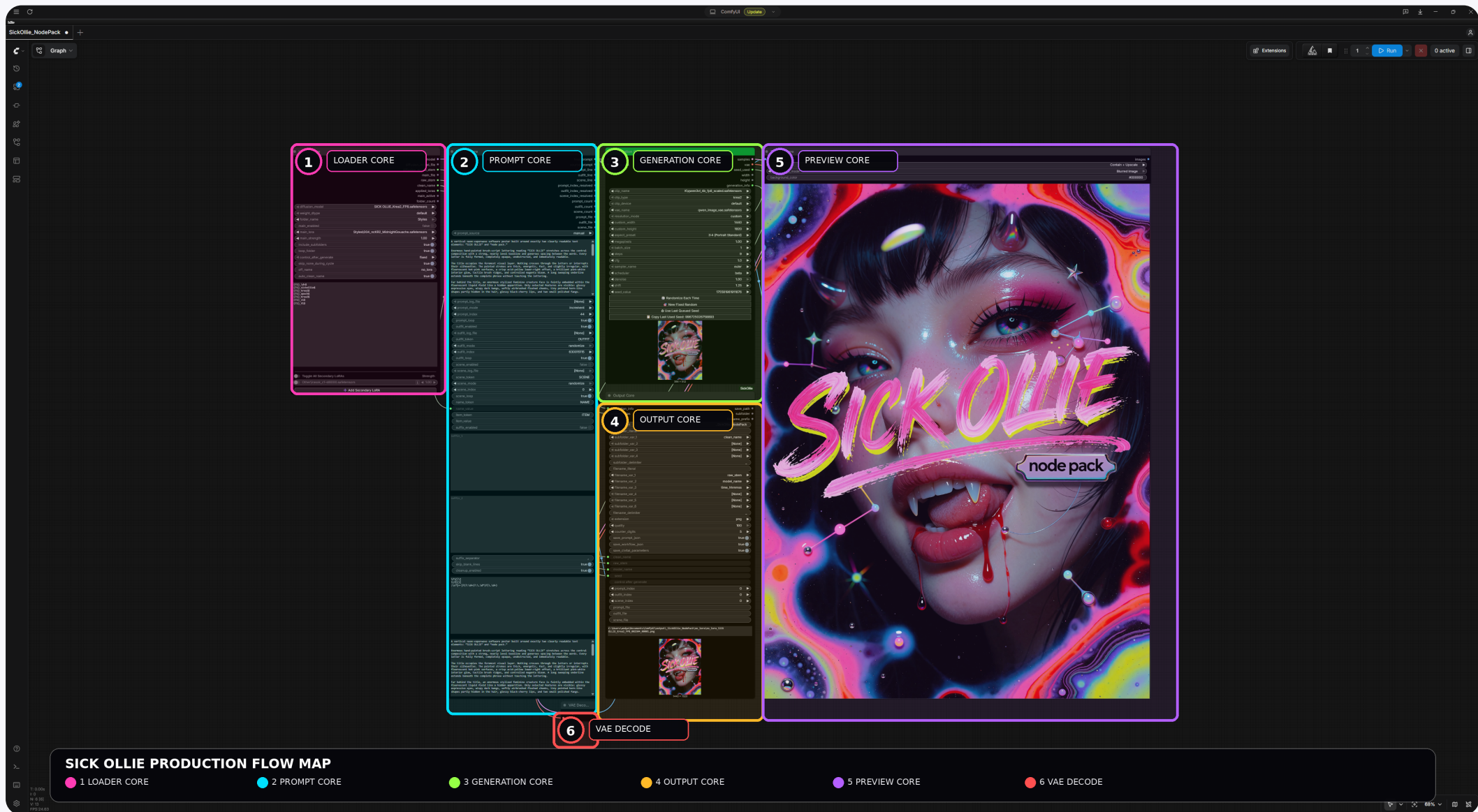
Copy the `SickOllieLogs` folder into the ComfyUI input directory so the final location becomes:

```
ComfyUI/input/SickOllieLogs/  
├─ prompts/  
├─ outfits/  
└─ scenes/
```

After copying, restart ComfyUI and hard-refresh the browser so Prompt Core rebuilds its file dropdowns.

Prima Tip: Copy the whole `SickOllieLogs` folder instead of moving individual text files. The included subfolders demonstrate how Prompt Core keeps large libraries organized while storing portable relative paths.

Quick-Start Wiring



Production flow map. The five custom production nodes surround the standard VAE Decode bridge. The color key is visual only; the underlying ComfyUI data types remain unchanged.

```

Loader Core model
  → Generation Core model

Loader Core clean_name
  → Prompt Core name_value

Loader Core clean_name
  → Output Core clean_name

Loader Core raw_stem
  → Output Core raw_stem

Loader Core diffusion_model_stem
  → Output Core model_name

Loader Core applied_loras
  → Output Core applied_loras

Prompt Core final_prompt
  → Generation Core positive_text

Generation Core samples
  → VAE Decode samples

Generation Core vae
  → VAE Decode vae

Generation Core seed_used
  → Output Core seed

Generation Core generation_info
  → Output Core generation_info

VAE Decode IMAGE
  → Output Core images

Output Core images
  → Preview Core images

```

Optional:

```

Prompt Core final_prompt
  → Persist + Display Resolved Prompt text

```

VAE Decode is a standard ComfyUI node and is not included in this pack.

Chapter 1: Loader Core

Category: Sick Ollie/LoRA

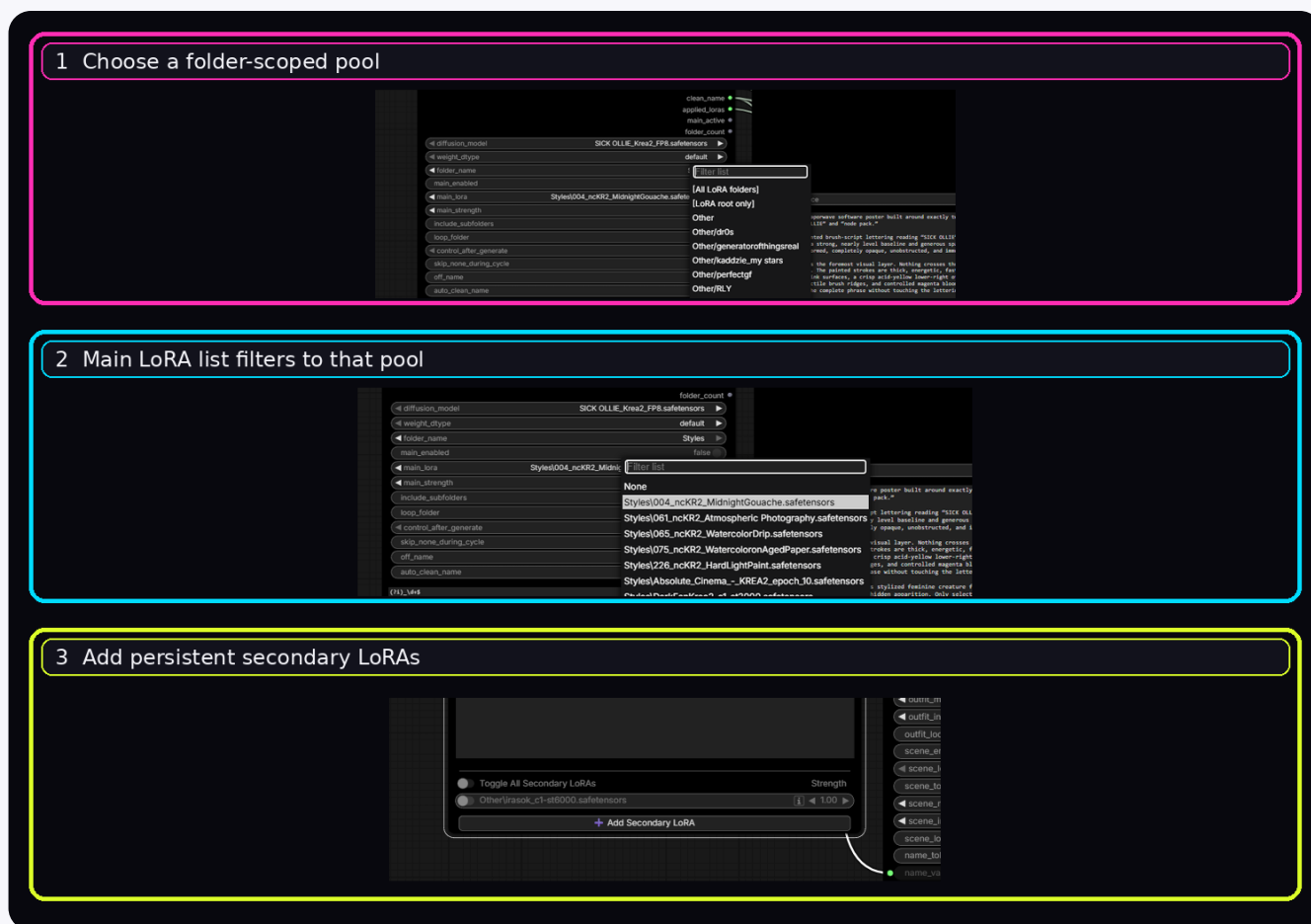
Loader Core is the front gate of the production system.

It performs four jobs:

1. loads the diffusion model
2. selects and applies the primary LoRA
3. applies any secondary LoRAs
4. converts machine-oriented filenames into strings useful elsewhere

The fourth job is what turns Loader Core from a loader into an automation engine.

Legendary Feature: Folder-Scoped LoRA Cycling



The folder-scoped testing lane. Choose a real LoRA subfolder, receive a filtered primary-LoRA pool, then keep persistent style or support LoRAs in the secondary stack.

Loader Core reads the user's actual ComfyUI LoRA directory and automatically builds its folder dropdown from that structure.

Suppose the LoRA directory looks like this:

```
loras/  
├─ becky_root_test.safetensors  
├─ People/  
│   ├─ becky_v1.safetensors  
│   ├─ becky_v2.safetensors  
│   └─ Rachel/  
│       ├─ rachel_epoch_10.safetensors  
│       └─ rachel_epoch_20.safetensors  
│   └─ Archive/  
│       └─ old_becky.safetensors  
├─ Styles/  
│   ├─ sick_light.safetensors  
│   └─ Fashion/  
│       └─ flash_editorial.safetensors  
└─ Experiments/  
    └─ strange_mix.safetensors
```

Loader Core generates choices resembling:

```
[All LoRA folders]  
[LoRA root only]  
Experiments  
People  
People/Archive  
People/Rachel  
Styles  
Styles/Fashion
```

Why parent folders appear

For every LoRA, Loader Core discovers its parent folder and every ancestor folder above it.

Because `People/Rachel` exists, both of these become selectable:

```
People  
People/Rachel
```

This lets one directory structure support broad and narrow testing pools.

[All LoRA folders]

Every LoRA visible to ComfyUI is eligible.

Use this for a full-library shuffle or when your library is already very small.

[LoRA root only]

Only LoRAs placed directly inside the LoRA root are eligible.

From the example above, this would include:

```
becky_root_test.safetensors
```

It would not include anything inside `People` , `Styles` , or `Experiments` .

Selecting a specific folder

Choose:

```
People/Rachel
```

The **main LoRA dropdown immediately filters itself** to:

```
None
People/Rachel/rachel_epoch_10.safetensors
People/Rachel/rachel_epoch_20.safetensors
```

Unrelated LoRAs disappear from that dropdown.

The full library is still available to the secondary LoRA chooser. The folder restriction belongs specifically to the primary testing lane.

Why This Exists: A main LoRA test should not suddenly wander from a Rachel identity into a lighting style, an archived Becky model, or a completely unrelated experiment.

Changing the folder

When `folder_name` or `include_subfolders` changes, Loader Core:

1. recalculates the eligible primary LoRAs
2. rewrites the main LoRA dropdown
3. automatically selects the first valid LoRA in the new pool

If the selected folder contains no eligible LoRAs, the main selection becomes:

```
None
```

`include_subfolders` off

The parent folder must match exactly.

Selecting:

```
People
```

includes:

```
People/becky_v1.safetensors
People/becky_v2.safetensors
```

It does not include:

```
People/Rachel/rachel_epoch_10.safetensors
People/Archive/old_bekky.safetensors
```

include_subfolders on

The exact selected folder and every folder beneath it become eligible.

Selecting:

```
People
```

now includes:

```
People/becky_v1.safetensors
People/becky_v2.safetensors
People/Rachel/rachel_epoch_10.safetensors
People/Rachel/rachel_epoch_20.safetensors
People/Archive/old_bekky.safetensors
```

Prima Tip: Keep subfolders off when each folder represents one clean testing set. Turn them on when a parent folder intentionally represents a whole collection.

The sealed cycling pool

The combination of:

```
folder_name          = People/Rachel
control_after_generate = increment
loop_folder           = on
skip_none_during_cycle = on
```

creates a sealed two-LoRA cycle:

```
rachel_epoch_10
↓
rachel_epoch_20
↓
rachel_epoch_10
↓
rachel_epoch_20
```

It cannot leak into Becky, Styles, Experiments, or the root folder.

That is the heart of the feature.

Queue-time progression

The selector advances **after each prompt is queued**, not after the previous image finishes rendering.

Suppose the main dropdown currently shows:

```
rachel_epoch_10
```

and the user queues four prompts.

The serialized queue becomes:

```
queued image 1 → rachel_epoch_10
queued image 2 → rachel_epoch_20
queued image 3 → rachel_epoch_10
queued image 4 → rachel_epoch_20
```

The first image receives the LoRA visible at the moment Queue is pressed. The dropdown then advances before the next prompt is serialized.

This means a large queue can contain different LoRAs before execution begins.

Common Trap: The control is named `control_after_generate` for familiarity, but the progression occurs after each prompt is queued. That queue-time behavior is what makes multi-image LoRA batches possible.

Progression modes

Mode	Behavior
<code>fixed</code>	Keeps the current primary LoRA.
<code>increment</code>	Moves forward through the filtered folder pool.
<code>decrement</code>	Moves backward through the filtered folder pool.
<code>randomize</code>	Selects a different eligible LoRA when more than one choice exists.

`loop_folder`

When enabled:

```
final LoRA → first LoRA
first LoRA → final LoRA when decrementing
```

When disabled, progression stops at the nearest endpoint.

skip_none_during_cycle

None always remains available for manual selection.

When this option is on, **None** is removed from automatic increment, decrement, and random pools.

This prevents a no-LoRA image from appearing during every folder loop.

When off, **None** becomes a real position in the cycle.

Random mode

Random mode attempts to choose a value different from the currently selected LoRA.

If the filtered folder contains only one eligible LoRA, it remains on that LoRA.

folder_count

The **folder_count** output reports the number of actual LoRA files eligible under the current folder and subfolder settings.

None is not counted.

Use it to confirm:

- the expected folder was found
- subfolder behavior is working
- the testing pool contains the expected number of files

Legendary Setup: Put every training checkpoint for one identity inside its own subfolder, select that subfolder, enable looping, and queue the number of checkpoints you want tested. Loader Core turns the folder itself into the batch plan.

The Three-Name System

Suppose the selected LoRA is:

```
People/Becky/becky_sickollie_krea2_epoch_12.safetensors
```

Loader Core produces:

```
main_file:
People/Becky/becky_sickollie_krea2_epoch_12.safetensors

raw_stem:
becky_sickollie_krea2_epoch_12

clean_name:
becky
```

main_file

The exact selected LoRA value, including its relative folder and extension.

Use it when exact source identification matters.

raw_stem

The basename without folder or extension.

It preserves training labels, versions, epochs, and project suffixes.

This is ideal for traceable filenames:

```
becky_sickollie_krea2_epoch_12
```

clean_name

The result of applying `cleanup_rules` to `raw_stem`.

This is intended for human-facing uses:

- replacing `NAME` in prompts
- tidy subject folders
- captions
- labels
- clean organization

```
becky
```

Name Cleanup and Regex Guide

The packaged default cleanup rules are:

```
(?i)_\d+$
(?i)_sickollie$
(?i)_krea2$
(?i)_epoch$
```

Loader Core applies every rule in order, then repeats the entire rule list until the name stops changing.

Starting value:

```
becky_sickollie_krea2_epoch_12
```

Pass 1

`(?i)_\d+$` removes `_12`:

```
becky_sickollie_krea2_epoch
```

`(?i)_epoch$` can now match the new ending:

```
becky_sickollie_krea2
```

Pass 2

`(?i)_krea2$` removes `_krea2`:

```
becky_sickollie
```

Pass 3

`(?i)_sickollie$` removes `_sickollie`:

```
becky
```

Pass 4

Nothing changes, so cleanup stops.

This repeating behavior allows stacked suffixes to peel away one layer at a time.

After cleanup, leading and trailing spaces, underscores, hyphens, and periods are trimmed.

Rule format

Each nonblank line is one regular-expression rule.

Remove a match

A pattern without `=>` removes every match:

```
(?i)_krea2$
```

Replace a match

Use `=>` to replace instead:

```
[ -]+ => _
```

This converts runs of spaces or hyphens into underscores.

Comments

Lines beginning with `#` are ignored:

```
<h1 id="remove-my-training-version-suffix" class="chapter">Remove my training-version suffix</h1>
(?i)_v\d+$
```

Invalid rules

Invalid regex lines are ignored and reported in the ComfyUI console rather than crashing the generation.

Case sensitivity

Regex is case-sensitive unless `(?i)` is used.

Case-sensitive:

```
_krea2$
```

Matches:

```
becky_krea2
```

Does not match:

```
becky_Krea2
```

Case-insensitive:

```
(?i)_krea2$
```

Matches both.

Common Trap: Loader cleanup regex can be made case-insensitive. Prompt Core token replacement is literal and case-sensitive. These are two different systems.

Regex key

| Symbol | Meaning | Example | |---|---|---| | `(?i)` | Ignore uppercase/lowercase differences | `(?i)_krea2$` | | `^` | Beginning of the name | `(?i)^character_` | | `$` | End of the name | `(?i)_final$` | | `\d` | One digit | `\d` | | `\d+` | One or more digits | `(?i)_v\d+$` | | `.` | Any character | Use carefully | | `.*` | Any number of characters | Use very carefully | | `[ _-]` | One space, underscore, or hyphen | `[ _-]+` | | `+` | One or more of the previous item | `\d+` | | `?` | Makes the previous item optional | `-?` | | `(a|b)` | Match either `a` or `b` | `(draft|final)` | | `(?:...)` | Non-capturing group | `(?:v|version)` | | `\.` | Literal period | `\.safetensors$` | | `=>` | Sick Ollie replacement separator | `[ -]+ => _` |

Cleanup recipes

Remove `_v1`, `_v2`, `_v17`, and similar versions

```
(?i)_v\d+$
```

Remove a fixed `S0_` prefix

```
(?i)^S0_
```

Remove `-final`

```
(?i)-final$
```

Remove `_step6000`

```
(?i)_step\d+$
```

Remove `_st6000`

```
(?i)_st\d+$
```

Convert spaces and hyphens into underscores

```
[ -]+ => _
```

Convert underscores into spaces

```
_+ =>
```

There is a literal space after `=>`.

Remove several endings with one rule

```
(?i)_(final|trained|release)$
```

Example custom stack

Filename:

```
S0-Becky Krea2 v3 step6000.safetensors
```

Rules:

```
(?i)^S0[-_]*  
(?i)[-_]+step\d+$  
(?i)[-_]+v\d+$  
(?i)[-_]+krea2$  
[-_]+ => _
```

Result:

```
Becky
```

The remaining capitalization is preserved because cleanup removes and replaces text. It does not automatically lowercase the result.

Loader Core Controls

`diffusion_model`

Selects the base diffusion model loaded before any LoRAs are applied.

Files come from ComfyUI's configured `diffusion_models` locations.

`weight_dtype`

Controls the dtype requested while loading the diffusion model.

Value	Purpose
<code>default</code>	Uses the model and ComfyUI's normal loading behavior.
<code>fp8_e4m3fn</code>	Forces FP8 E4M3FN loading.
<code>fp8_e4m3fn_fast</code>	Uses FP8 E4M3FN with ComfyUI's FP8 optimizations.
<code>fp8_e5m2</code>	Forces FP8 E5M2 loading.

For a model already stored in a specialized or quantized format, `default` is usually the safest starting point.

`folder_name`

Defines the filtered primary-LoRA testing pool.

See [Legendary Feature: Folder-Scoped LoRA Cycling](#).

`main_enabled`

Controls whether the selected primary LoRA is applied.

The primary LoRA is active only when all three conditions are true:

- `main_enabled` is on
- `main_lora` is not `None`
- `main_strength` is not `0`

`main_lora`

The primary LoRA selected from the folder-filtered dropdown.

This is the only LoRA that participates in automatic folder progression.

`main_strength`

The strength applied to the primary LoRA.

A strength of `0` makes the LoRA inactive even when selected and enabled.

Negative strengths are permitted for models and workflows that intentionally use them.

`include_subfolders`

Controls whether nested folders beneath `folder_name` join the primary pool.

`loop_folder`

Controls whether incrementing and decrementing wrap at the ends of the filtered pool.

`control_after_generate`

Controls queue-time primary-LoRA progression:

- fixed
- increment
- decrement
- randomize

`skip_none_during_cycle`

Keeps `None` available manually but removes it from automatic progression.

`off_name`

The value emitted by `main_file`, `raw_stem`, and `clean_name` when the primary LoRA is inactive.

Default:

```
no_lora
```

This prevents downstream folders and filenames from becoming blank.

auto_clean_name

- on: applies `cleanup_rules` to `raw_stem`
- off: makes `clean_name` equal to `raw_stem`

Disable it when the filenames already contain exactly the desired human-facing name.

cleanup_rules

The editable regex system described above.

Rules run sequentially and repeat until stable, up to 20 passes.

Inactive metadata scrubbing

A selected LoRA can remain visible on the live canvas while being disabled.

Loader Core prevents that inactive selection from being falsely recorded as an applied resource in the queued prompt and saved workflow metadata.

This applies when:

- the primary LoRA is disabled
- the primary strength is `0`
- a secondary LoRA row is off
- a secondary strength is `0`

The live widget remains untouched for convenience, but the saved metadata reflects what actually affected the model.

Why This Exists: Selecting a LoRA and using a LoRA are not the same event. Saved metadata should describe the image, not merely the state of the dropdown.

Secondary LoRA Stack

Secondary LoRAs are applied after the primary LoRA.

They do not participate in folder cycling.

Use them for persistent support components such as:

- style
- lighting
- clothing
- detail enhancement
- a secondary identity

- correction behavior

Each row stores:

- on/off state
- LoRA selection
- strength

Available row actions include:

- Show Info
- Toggle On/Off
- Move Up
- Move Down
- Remove
- Add Secondary LoRA

The header toggle switches all secondary rows together.

Order can matter because LoRAs are applied sequentially.

Loader Core uses rgthree-comfy for its enhanced LoRA chooser and information tools, including:

- Civitai lookup
- trained words
- trigger words
- showcase images
- example prompts
- custom row interaction

Loader Core Outputs

`model`

The diffusion model after the active primary and secondary LoRAs have been applied.

Connect to:

```
Generation Core model
```

`diffusion_model_file`

The exact diffusion-model dropdown value, including relative folder and extension.

Example:

```
Krea2/SICK_OLLIE_Krea2_FP8.safetensors
```

Use this when exact source identification is needed.

diffusion_model_stem

The diffusion model basename without folder or extension.

Example:

```
SICK OLLIE_Krea2_FP8
```

Common connection:

```
Output Core model_name
```

main_file

When active, the exact primary LoRA path and filename.

When inactive, the `off_name`.

Example:

```
People/Becky/becky_sickollie_krea2_epoch_12.safetensors
```

raw_stem

The primary LoRA basename without folder or extension.

Example:

```
becky_sickollie_krea2_epoch_12
```

Common connection:

```
Output Core raw_stem
```

This preserves exact checkpoint traceability in output filenames.

clean_name

The human-friendly result of applying `cleanup_rules` to `raw_stem`.

Example:

```
becky
```

Common connections:

```
Prompt Core name_value  
Output Core clean_name
```

applied_loras

A newline-separated record containing only the LoRAs that actually affected the model and their strengths.

Example:

```
People/Becky/becky_sickollie_krea2_epoch_12.safetensors@1  
Styles/sick_light.safetensors@0.8
```

Connect to:

```
Output Core applied_loras
```

Output Core uses this to build LoRA tags and resource hashes.

main_active

Boolean status of the primary LoRA.

True only when the primary is enabled, selected, and has nonzero strength.

Useful for custom switches, logic, or status displays.

folder_count

The number of actual LoRA files eligible under the current `folder_name` and `include_subfolders` settings.

`None` is not counted.

Chapter 2: Prompt Core

Category: Sick Ollie/Prompt

Prompt Core turns reusable text libraries into a queueable prompt-production system.

A prompt log is a plain `.txt` file where each line is one complete reusable entry.

Three independent libraries are supported:

- prompts
- outfits
- scenes

Prompt Core selects one line from each enabled library and combines them through literal token replacement.

What Is a Prompt Log?

A prompt log can look like this:

```
A harsh-flash fashion photograph of NAME wearing OUTFIT beside a motel pool.  
A glossy studio portrait of NAME wearing OUTFIT against a cobalt seamless.  
A grainy convenience-store snapshot of NAME in OUTFIT, SCENE.
```

Each line is one prompt.

A prompt log is not JSON, CSV, or a numbered database. It is intentionally plain text because plain text is:

- portable
- easy to edit
- easy to search
- easy to generate
- easy to reuse
- friendly to automation

Making Your Own Logs

Prompt Core automatically uses:

```
ComfyUI/input/SickOllieLogs/  
├─ prompts/  
├─ outfits/  
└─ scenes/
```

Subfolders are allowed:

```
SickOllieLogs/  
├─ prompts/  
│   └─ fashion/  
│       └─ american_apparel.txt  
│       └─ posters/  
│           └─ sick_dolls.txt  
├─ outfits/  
│   └─ OUTFITS_FULL.txt  
└─ scenes/  
    └─ interiors.txt
```

The node stores portable relative values such as:

```
prompts/posters/sick_dolls.txt
```

It does not store the operating-system username or full ComfyUI installation path in the file dropdowns.

Absolute paths and `..` traversal are rejected.

Create a log

1. Open the appropriate folder.
2. Create a `.txt` file.
3. Put one complete entry on each line.
4. Save as UTF-8 when possible.
5. Restart ComfyUI and hard-refresh after adding, removing, or renaming files.

Prompt Core can also read:

- UTF-8 with BOM
- Windows CP1252
- Latin-1

Prompt-log example

```
A direct-flash bedroom fashion advertisement featuring NAME wearing OUTFIT.  
A cinematic roadside portrait of NAME wearing OUTFIT beneath sodium-vapor lights.  
A clean catalog photograph of NAME wearing OUTFIT against a white seamless.
```

Outfit-log example

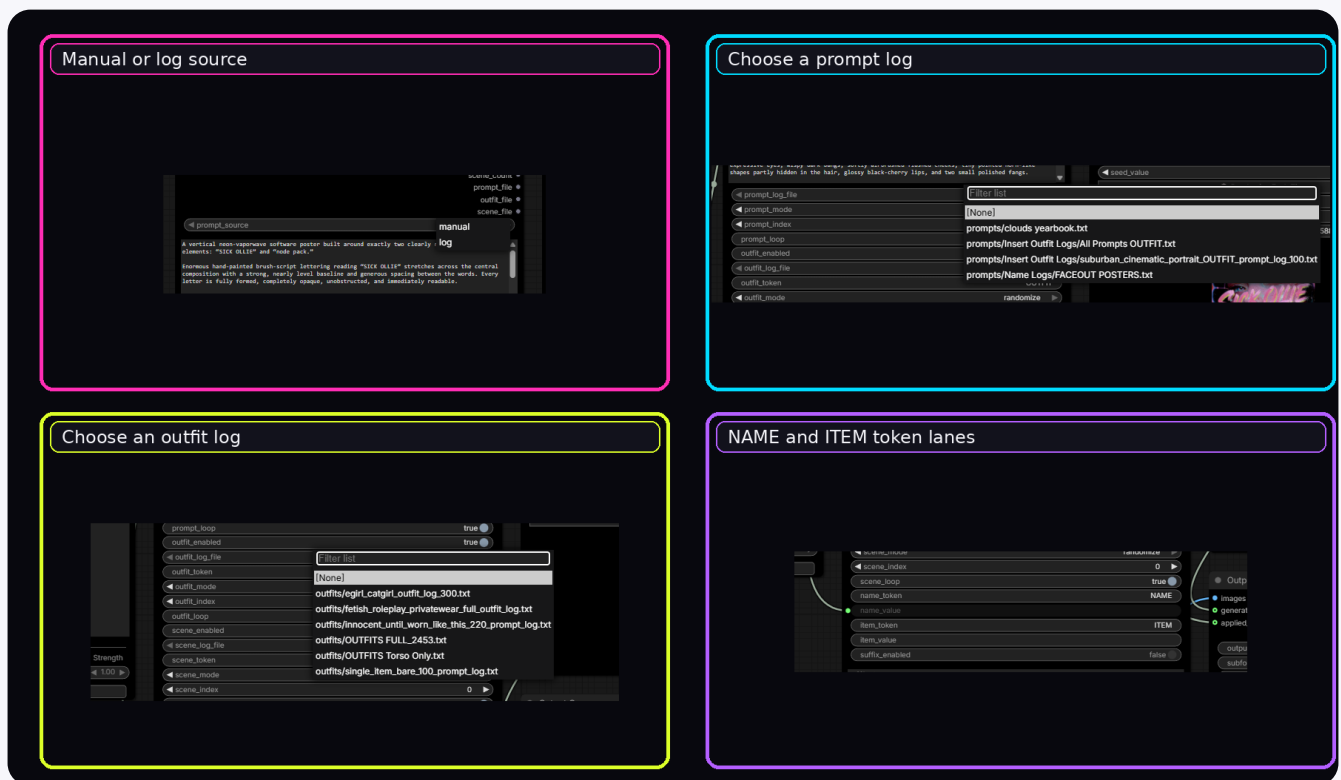
```
a pink terry-cloth bandeau top with matching shorts  
an oversized white tank with striped socks  
a cobalt babydoll dress with white sneakers
```

Scene-log example

rumpled white bedding and a chrome bedside lamp
an empty fluorescent laundromat at midnight
a sunlit apartment kitchen with yellow cabinets

Do not add numbering unless the numbering should become part of the generated prompt.

The NAME Pipeline



Prompt Core in four moves. Choose manual or log mode, select reusable prompt and outfit libraries, then let connected values replace literal `NAME`, `OUTFIT`, `SCENE`, and `ITEM` tokens.

This is one of the pack's central automation paths.

Step 1: Loader Core sees the LoRA

People/Becky/becky_sickollie_krea2_epoch_12.safetensors

Step 2: Loader Core creates `clean_name`

becky

Step 3: Connect the nodes

```
Loader Core clean_name  
→ Prompt Core name_value
```

Step 4: Write a prompt containing the literal token

```
A direct-flash advertisement featuring NAME wearing OUTFIT.
```

Step 5: Prompt Core replaces every exact NAME

```
A direct-flash advertisement featuring becky wearing OUTFIT.
```

Step 6: Outfit replacement occurs

```
A direct-flash advertisement featuring becky wearing a pink terry-cloth bandeau top with matching shorts.
```

That is why `clean_name` exists. The selected LoRA can identify itself to the prompt system.

The “That’s Wild” Moment: Select a different primary LoRA and the name inside every queued prompt changes automatically, provided its filename cleanup rules produce the desired name.

Token matching is literal and case-sensitive

These are three different tokens:

```
NAME  
Name  
name
```

The packaged default is:

```
NAME
```

The prompt must contain exactly the value written in `name_token`.

All exact occurrences are replaced.

If `name_value` is empty, name replacement is skipped and the token remains in the prompt.

The same rules apply to:

- `OUTFIT`
- `SCENE`
- `ITEM`

Common Trap: Adding `(?i)` to Loader Core cleanup can fix filename capitalization. It does not affect Prompt Core's token matching. Change the token field or use the exact token in the prompt.

Prompt Progression

Prompt, outfit, and scene logs each have their own:

- mode
- raw index
- loop setting

Queue-time behavior

The first queued prompt uses the current index.

After that prompt is queued, Prompt Core advances the raw index for the next queued prompt.

A queue of four prompts can therefore serialize four different lines before generation begins.

Modes

Mode	Behavior
<code>fixed</code>	Keeps the current raw index.
<code>increment</code>	Adds <code>1</code> after each queued prompt.
<code>decrement</code>	Subtracts <code>1</code> after each queued prompt.
<code>randomize</code>	Replaces the raw index with a large random integer.

Prompt progression is active only when `prompt_source` is `log`.

Outfit progression is active only when `outfit_enabled` is on.

Scene progression is active only when `scene_enabled` is on.

Raw index versus resolved index

Prompt Core keeps the raw index separate from the actual line selected.

With looping on, the resolved index uses modulo arithmetic.

For a 2,453-line outfit log:

```
raw index 0      → resolved index 0      → line 1
raw index 2452   → resolved index 2452   → line 2453
raw index 2453   → resolved index 0      → line 1
raw index 2454   → resolved index 1      → line 2
raw index -1     → resolved index 2452   → line 2453
```

This is why a huge randomized raw index is harmless when looping is enabled.

With looping off, the index is clamped:

```
negative raw index → first line
too-large raw index → final line
```

Prima Tip: Keep loop on for randomize mode. A huge random raw index combined with loop off usually clamps to the final line.

Prompt Core Controls

prompt_source

Value	Behavior
manual	Uses <code>manual_prompt</code> as the base prompt.
log	Uses the selected line from <code>prompt_log_file</code> .

The unused source remains stored but does not become `source_prompt`.

manual_prompt

The base prompt used in manual mode.

Manual mode still supports:

- name replacement
- outfit replacement
- scene replacement
- item replacement
- suffixes
- cleanup rules

This is useful for testing one prompt while retaining the rest of the automation.

prompt_log_file

Selects a `.txt` file under:

```
SickOllieLogs/prompts/
```

`prompt_mode`

Controls queue-time progression of `prompt_index` when log mode is active.

`prompt_index`

The raw zero-based prompt index.

Line 1 is index `0`.

`prompt_loop`

- on: resolves the raw index with modulo
- off: clamps to the first or final line

`outfit_enabled`

Enables outfit-log selection and replacement.

When off:

- `OUTFIT` replacement is skipped
- outfit progression stops
- the outfit line and count outputs can still report the selected file's state

`outfit_log_file`

Selects a `.txt` file under:

```
SickOllieLogs/outfits/
```

`outfit_token`

The literal text replaced inside the assembled prompt.

Default:

```
OUTFIT
```

`outfit_mode`

Controls queue-time outfit-index progression.

`outfit_index`

The raw zero-based outfit index.

outfit_loop

Controls modulo wrapping or endpoint clamping.

scene_enabled

Enables scene-log selection and replacement.

scene_log_file

Selects a `.txt` file under:

```
SickOllieLogs/scenes/
```

scene_token

The literal scene token.

Default:

```
SCENE
```

scene_mode

Controls queue-time scene-index progression.

scene_index

The raw zero-based scene index.

scene_loop

Controls modulo wrapping or endpoint clamping.

name_token

The literal name token searched inside the assembled prompt.

Default:

```
NAME
```

name_value

The replacement inserted wherever `name_token` appears.

Recommended connection:

```
Loader Core clean_name  
→ Prompt Core name_value
```

item_token

A general-purpose literal token.

Default:



ITEM

Use it for props, products, objects, vehicles, foods, or any other one-off replacement system.

item_value

The replacement for `item_token`.

This can be typed manually or connected from another string-producing node.

suffix_enabled

Enables the suffix system.

suffix_1 and suffix_2

Optional text blocks appended after token replacement.

Use them for reusable additions such as:

- camera treatment
- lighting style
- model trigger phrases
- quality language
- a constant text-layout requirement

Blank suffixes are ignored.

suffix_separator

Joins:

- the assembled prompt to the suffix text
- `suffix_1` to `suffix_2`

Default:



,

If the base prompt is empty, the suffix becomes the prompt without a leading separator.

skip_blank_lines

When on, blank lines are removed before:

- counting entries
- resolving indexes
- selecting lines

When off, a blank line is a valid entry.

cleanup_enabled

Controls whether Prompt Core's final regex cleanup runs.

cleanup_rules

One regex per line.

Like Loader Core:

- pattern alone removes matches
- `PATTERN => REPLACEMENT` replaces matches
- blank lines and `#` comments are ignored
- rules repeat until stable
- invalid rules are ignored and reported in the console

saved_prompt

A read-only display of the most recently executed final prompt.

Prompt Core stores the resolved prompt in the workflow properties written into saved PNG metadata. Reloading the PNG restores this display.

This field does not control generation. It is a receipt.

Prompt Cleanup

The packaged defaults are designed to flatten common weighted-prompt notation into plain natural language:

```
\?\[|\]  
\?[(|)]  
:s*[+-]?(?:\d+(?:\.\d*)?|\.\d+)
```

What the defaults remove

Input fragment	Result
(portrait:1.2)	portrait
(dramatic lighting)	dramatic lighting
?[subject]	subject
keyword:-0.5	keyword

The rules remove:

- `?[`
- closing `]`
- parentheses, including optionally escaped parentheses
- a colon followed by a signed numeric weight

This is useful when importing weighted prompt logs into a model or workflow that prefers direct natural-language instructions.

Disable `cleanup_enabled` when those symbols should remain literal.

Prompt Core Outputs

`final_prompt`

The completed prompt after:

1. source selection
2. outfit replacement
3. scene replacement
4. name replacement
5. item replacement
6. suffix assembly
7. cleanup

Connect to:

Generation Core positive_text

`source_prompt`

The untouched base source selected before replacements.

- manual text in manual mode
- selected prompt-log line in log mode

Useful for debugging the difference between the original template and final prompt.

`prompt_line`

The selected line from `prompt_log_file`, even when manual mode is active.

Useful for logging or inspection.

`outfit_line`

The selected outfit-log line.

scene_line

The selected scene-log line.

prompt_index_resolved

The actual zero-based prompt line used after modulo or clamping.

Connect this to Output Core when the filename should record the real selected line.

outfit_index_resolved

The actual outfit index used.

scene_index_resolved

The actual scene index used.

prompt_count

Number of eligible prompt-log lines after blank-line handling.

outfit_count

Number of eligible outfit-log lines.

scene_count

Number of eligible scene-log lines.

prompt_file

The portable relative prompt-log path.

Example:

```
prompts/posters/sick_dolls.txt
```

outfit_file

The portable relative outfit-log path.

scene_file

The portable relative scene-log path.

These file outputs can feed Output Core, which can turn them into filename-safe stems.

Chapter 3: Generation Core

Category: Sick Ollie/Generation

Generation Core combines the usual text-encoder, prompt-conditioning, latent, model-shift, sampler, VAE, and seed path into one production node.

Its negative conditioning is intentionally an encoded empty string.

The node is currently built around a Krea2-oriented workflow while retaining several compatible CLIP-type choices.

Why Generation Core Exists

A normal graph can require separate nodes for:

- CLIP loading
- positive prompt encoding
- negative prompt encoding
- empty latent creation
- sampling shift
- KSampler
- VAE loading
- seed bookkeeping
- generation metadata assembly

Generation Core keeps that path consistent and returns the values Output Core needs to document the image accurately.

It caches the most recently used text encoder and VAE selection to avoid unnecessary reload work when those choices remain unchanged.

Generation Core Controls

`model`

The model to sample.

Recommended connection:

```
Loader Core model  
→ Generation Core model
```

positive_text

The final prompt to encode.

Recommended connection:

```
Prompt Core final_prompt  
→ Generation Core positive_text
```

clip_name

Selects the text-encoder file from ComfyUI's `text_encoders` locations.

clip_type

Tells ComfyUI how the selected text encoder should be interpreted.

Available values:

- krea2
- sd3
- stable_diffusion
- stable_cascade
- pixart
- flux
- default

For the intended Krea2 workflow, use:

```
krea2
```

clip_device

Value	Behavior
default	Lets ComfyUI manage loading and offloading.
cpu	Requests CPU loading and offloading.
gpu	Requests the active torch device for loading and offloading.

`default` is the safest general choice.

vae_name

Selects the VAE loaded internally.

Generation Core returns the loaded VAE as an output so the sampled latent can be decoded with a standard `VAE Decode` node.

resolution_mode

Value	Behavior
custom	Uses <code>custom_width</code> and <code>custom_height</code> .
preset	Calculates dimensions from <code>aspect_preset</code> and <code>megapixels</code> .

custom_width

Width used in custom mode.

The UI steps in multiples of 8.

custom_height

Height used in custom mode.

aspect_preset

Available preset ratios:

- 1:1
- 2:3
- 3:4
- 4:5
- 9:16
- 4:3
- 3:2
- 16:9

megapixels

Approximate total pixel count used in preset mode.

Generation Core calculates dimensions that preserve the chosen ratio, then rounds them to multiples of 8.

Example:

```
aspect = 3:4  
megapixels = 1.0
```

produces dimensions near one million total pixels while preserving 3:4.

batch_size

Number of latent images sampled in one execution.

This differs from ComfyUI's Queue count:

- Queue count creates multiple queued executions
- batch size creates multiple images inside one execution

steps

Number of sampling steps.

cfg

Classifier-free guidance value passed to the sampler.

sampler_name

Selects one of the samplers installed and exposed by ComfyUI.

scheduler

Selects one of ComfyUI's available schedulers.

denoise

Sampling denoise strength.

For a fresh empty latent, **1.0** is the normal full-generation setting.

shift

Applies ComfyUI's AuraFlow model-sampling shift patch before sampling.

This is recorded as **Model shift** in Output Core metadata.

seed

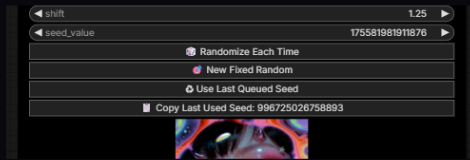
The backend field is internally named **seed_value** for frontend stability but appears as **seed**.

Valid behavior:

- **-1** requests a new random seed during execution
- **0** or greater is treated as a fixed seed
- fixed seeds are clamped to the supported range

Seed System

Seed controls: explore, freeze, restore, copy



Exploration without seed amnesia. Randomize freely, mint a fixed seed, restore the last queued result, or copy the exact seed without altering the active generation state.

Randomize Each Time

Sets the active seed field to:

-1

Every execution resolves **-1** into a new actual seed.

New Fixed Random

Generates a random seed immediately and places it into the seed field.

The value remains fixed until changed.

Use Last Queued Seed

Places the most recently executed seed back into the active seed field.

Use this to reproduce the last randomized result.

Copy Last Used Seed

Copies the most recently executed seed without changing the active seed field.

The displayed last-used seed persists through saved workflow and PNG reloads.

Prima Tip: Leave the active seed at **-1** for exploration. When a result lands, use **Use Last Queued Seed** to convert that exact result into a fixed reproducible setup.

Generation Core Outputs

samples

The sampled latent.

Connect to:

VAE Decode samples

vae

The loaded VAE.

Connect to:

VAE Decode vae

seed_used

The exact nonnegative seed used for sampling.

When the active seed was `-1`, this output reveals the random seed actually chosen.

Connect to:

Output Core seed

width

The resolved generation width.

This is especially useful in preset mode where the final dimensions are calculated.

height

The resolved generation height.

generation_info

A JSON string containing:

- positive prompt
- empty negative prompt
- text encoder name
- CLIP type
- CLIP device
- VAE name
- resolution mode
- width

- height
- batch size
- exact seed
- steps
- CFG
- sampler
- scheduler
- denoise
- model shift

Connect to:

Output Core generation_info

This is the handoff that lets Output Core describe what Generation Core actually did rather than guessing from visible widgets.

Chapter 4: Output Core

Category: Sick Ollie/Output

Output Core is both a file builder and a metadata recorder.

It creates:

```
output root / constructed subfolder / constructed filename_counter.extension
```

while preserving enough information to reload the workflow and identify the resources used.

Why Output Core Exists

A useful output filename should answer questions without becoming an essay:

- which subject?
- which exact LoRA checkpoint?
- which model?
- which prompt log?
- which seed?
- when was it rendered?

Different jobs need different answers.

That is why Output Core offers both:

```
clean_name  
raw_stem
```

A tidy folder can use:

```
becky
```

while the file itself preserves:

```
becky_sickollie_krea2_epoch_12
```

Example:

```
output/  
└─ _SickOllie_Art/  
    └─ becky/  
        └─ becky_sickollie_krea2_epoch_12_SICK_OLLIE_Krea2_FP8_194939_00001.png
```

The folder is easy to browse. The filename remains traceable.

Folder and Filename Construction

Subfolder builder

Output Core combines:

1. `subfolder_literal`, when nonblank
2. `subfolder_var_1`
3. `subfolder_var_2`
4. `subfolder_var_3`
5. `subfolder_var_4`

Nonempty pieces are joined with `subfolder_delimiter`.

Filename builder

Output Core combines:

1. `filename_literal`, when nonblank
2. `filename_var_1`
3. `filename_var_2`
4. `filename_var_3`
5. `filename_var_4`
6. `filename_var_5`
7. `filename_var_6`

Nonempty pieces are joined with `filename_delimiter`.

`[None]` choices are skipped.

If no filename piece remains, the prefix becomes:

```
image
```

ComfyUI's standard save-path helper then appends the counter.

Sanitization

Each component is cleaned for filesystem safety.

Characters such as these become underscores:

```
< > : " / \ | ? *
```

Control characters are removed, and trailing spaces or periods are trimmed.

Output Variable Key

Variable	Source and purpose
[None]	Skip this variable slot.
clean_name	Human-friendly Loader Core identity name, such as <code>becky</code> .
raw_stem	Exact primary LoRA basename, such as <code>becky_krea2_epoch_12</code> .
model_name	Connected model-name string or auto-detected upstream model stem.
seed	Exact connected seed, normally Generation Core <code>seed_used</code> .
prompt_index	Connected prompt index. Prefer Prompt Core <code>prompt_index_resolved</code> .
outfit_index	Connected outfit index. Prefer Prompt Core <code>outfit_index_resolved</code> .
scene_index	Connected scene index. Prefer Prompt Core <code>scene_index_resolved</code> .
prompt_file_stem	Filename stem extracted from connected <code>prompt_file</code> .
outfit_file_stem	Filename stem extracted from connected <code>outfit_file</code> .
scene_file_stem	Filename stem extracted from connected <code>scene_file</code> .
date_yyyymmdd	Current date, such as <code>20260726</code> .
date_mmdd	Current month and day, such as <code>0726</code> .
time_hhmm	Current 24-hour time, such as <code>1949</code> .
time_hhmmss	Current 24-hour time with seconds, such as <code>194939</code> .
datetime_yyyymmdd_hhmmss	Full date and time, such as <code>20260726_194939</code> .

Recommended index connections

Use Prompt Core's resolved indexes:

```
Prompt Core prompt_index_resolved
→ Output Core prompt_index

Prompt Core outfit_index_resolved
→ Output Core outfit_index

Prompt Core scene_index_resolved
→ Output Core scene_index
```

This records the actual line selected after modulo or clamping, not an enormous randomized raw index.

Recommended file connections

```
Prompt Core prompt_file
  → Output Core prompt_file

Prompt Core outfit_file
  → Output Core outfit_file

Prompt Core scene_file
  → Output Core scene_file
```

Output Core converts:

```
prompts/posters/sick_dolls.txt
```

into:

```
sick_dolls
```

when `prompt_file_stem` is chosen as a variable.

Naming recipes

Clean subject folders with exact checkpoint filenames

```
output_root      = _SickOllie_Art
subfolder_var_1  = clean_name

filename_var_1   = raw_stem
filename_var_2   = model_name
filename_var_3   = time_hhmmss
```

Dataset testing with prompt and outfit indexes

```
subfolder_var_1  = clean_name

filename_var_1   = raw_stem
filename_var_2   = prompt_index
filename_var_3   = outfit_index
filename_var_4   = seed
```

Prompt-log archive

```
subfolder_var_1  = prompt_file_stem
subfolder_var_2  = clean_name

filename_var_1   = raw_stem
filename_var_2   = scene_file_stem
filename_var_3   = datetime_yyyymmdd_hhmmss
```

Metadata and Civitai Resource Detection

Output Core searches upstream through the saved workflow graph for the nearest recognized model loader, including:

- Loader Core
- UNET Loader
- Checkpoint Loader
- Checkpoint Loader Simple
- UNET Loader GGUF

In the intended workflow, it finds Loader Core automatically.

It then attempts to locate and hash:

- the diffusion model
- the selected VAE
- every active LoRA listed by `applied_loras`

Why `applied_loras` matters

Loader Core sends entries such as:

```
People/Becky/becky_epoch_12.safetensors@1  
Styles/sick_light.safetensors@0.8
```

Output Core parses the file path and strength, then creates:

```
<lora:becky_epoch_12:1>  
<lora:sick_light:0.8>
```

It also calculates resource hashes when the local files can be found.

Metadata can include

- Positive prompt
- Negative prompt
- Steps
- Sampler
- CFG scale
- Seed
- Size
- Model
- Model hash
- VAE
- VAE hash
- Denoising strength when not `1.0`
- Model shift
- LoRA hashes

- consolidated Hashes JSON
- native ComfyUI prompt JSON
- native ComfyUI workflow JSON
- Sick Ollie resolved prompt fields

Hash cache

Resource hashes are cached inside the pack's `.cache` directory and reused while the file's modification time remains unchanged.

This avoids hashing very large model files on every save.

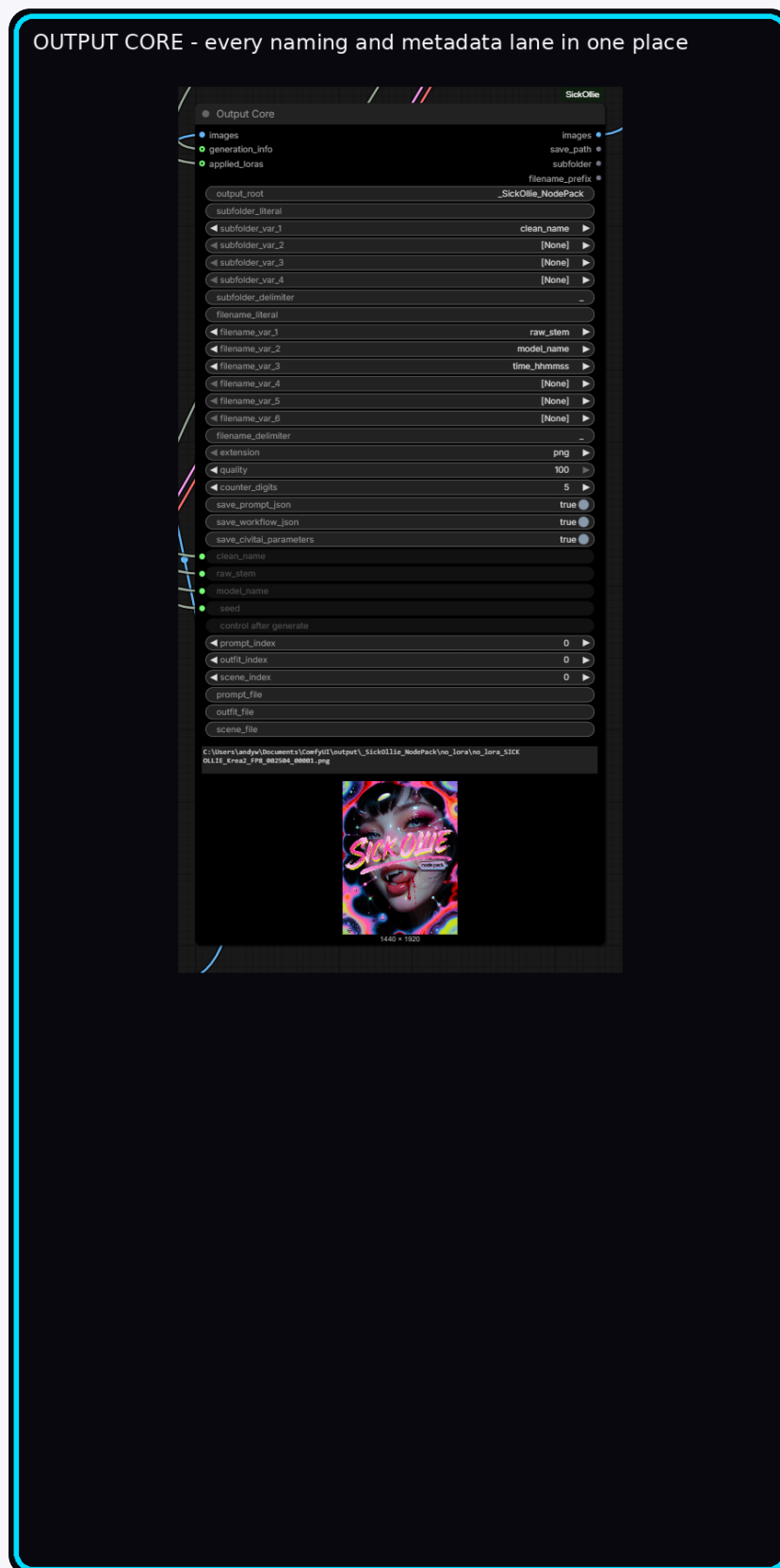
Civitai matching

A renamed file can still match because the hash comes from its contents.

A merged, converted, retrained, or differently quantized file may have a different hash and therefore may not match an existing Civitai model version.

Why This Exists: Metadata should identify the files that truly shaped the image, not require the creator to type those resources again by hand.

Output Core Controls



Output Core as a production ledger. Folder construction, filename variables, metadata switches, connected identity values, log indexes, resource paths, and the final saved path remain visible in one place.

images

The image or image batch to save.

Recommended connection:

```
VAE Decode IMAGE  
→ Output Core images
```

output_root

The root folder relative to ComfyUI's configured output directory.

Example:

```
_Sick0llie_Art
```

This is not an absolute path.

subfolder_literal

Optional fixed text placed first in the generated subfolder name.

subfolder_var_1 through subfolder_var_4

Choose up to four dynamic subfolder components.

subfolder_delimiter

Joins the subfolder pieces.

Default:

```
_
```

filename_literal

Optional fixed text placed first in the filename prefix.

filename_var_1 through filename_var_6

Choose up to six dynamic filename components.

filename_delimiter

Joins filename pieces.

Default:

```
_
```

extension

Available formats:

- png
- jpg
- webp

quality

Used for JPG and WebP quality.

PNG ignores this value.

counter_digits

Controls the zero-padded counter width.

Value:

```
5
```

produces:

```
00001
```

save_prompt_json

Embeds ComfyUI's prompt JSON metadata.

save_workflow_json

Embeds the workflow JSON required for dragging the image back into ComfyUI.

save_civitai_parameters

Builds and embeds the Civitai-style parameter block and resource fields.

clean_name

String source used by the `clean_name` naming variable.

Recommended connection:

```
Loader Core clean_name  
→ Output Core clean_name
```

raw_stem

String source used by the `raw_stem` naming variable.

model_name

String source used by the `model_name` naming variable.

If empty, Output Core attempts to use the automatically detected upstream model stem.

Connecting Loader Core's output makes naming explicit and predictable:

```
Loader Core diffusion_model_stem  
→ Output Core model_name
```

seed

Integer source used by the `seed` naming variable and metadata fallback.

Recommended connection:

```
Generation Core seed_used  
→ Output Core seed
```

prompt_index

Integer source used by the `prompt_index` naming variable.

outfit_index

Integer source used by the `outfit_index` naming variable.

scene_index

Integer source used by the `scene_index` naming variable.

prompt_file

Path source used by `prompt_file_stem`.

outfit_file

Path source used by `outfit_file_stem`.

scene_file

Path source used by `scene_file_stem`.

saved_path

Read-only display of the most recently saved absolute file path.

The value persists when the saved workflow is reloaded.

generation_info

Optional forced string input.

Recommended connection:

```
Generation Core generation_info  
→ Output Core generation_info
```

This supplies the exact settings used during sampling.

applied_loras

Optional forced string input.

Recommended connection:

```
Loader Core applied_loras  
→ Output Core applied_loras
```

This supplies the exact active LoRA stack for tags and hashes.

Output Core Outputs

images

The original input image tensor passed through unchanged.

Common connection:

```
Output Core images  
→ Preview Core images
```

save_path

The absolute path of the final image saved in this execution.

For an image batch, this is the path of the final saved image.

subfolder

The generated subfolder segment, excluding `output_root`.

filename_prefix

The generated filename prefix before the automatic counter and extension.

Chapter 5: Preview Core

Category: Sick Ollie/Output

Preview Core displays an image inside a resizable node while returning the original image unchanged.

It does not resize the tensor, crop the saved image, or alter pixels.

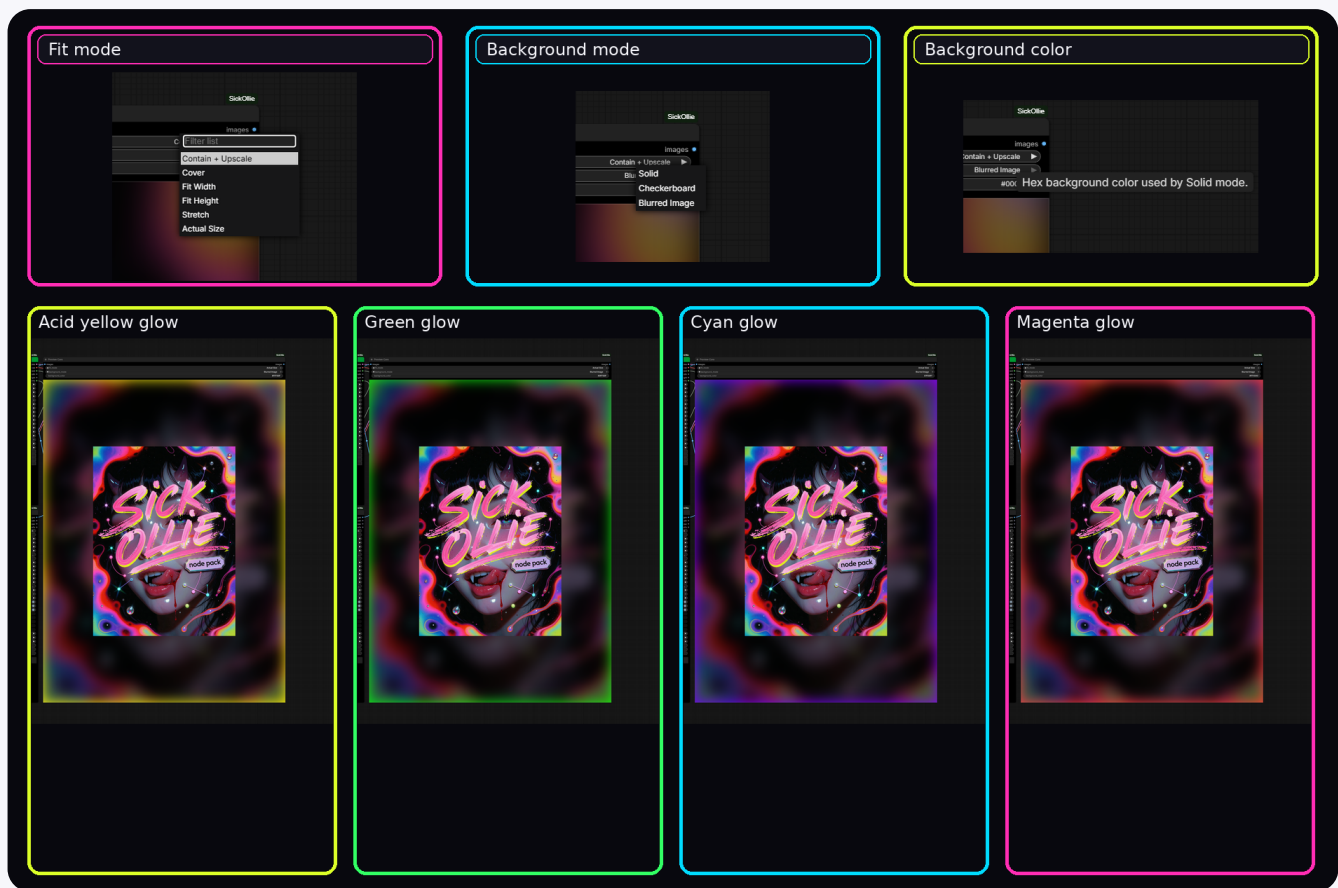
Why Preview Core Exists

Standard preview nodes can leave a large unused black area when the node frame and image have different sizes.

Preview Core provides deliberate display modes so the canvas preview can prioritize:

- seeing the whole image
- filling the frame
- fitting one dimension
- preserving actual pixel size
- using a more attractive background

Preview Core Controls



Preview with pizazz. Fit mode controls the foreground image. Background mode controls the empty frame. In Blurred Image mode, the chosen background color tints the blurred halo, which is why the same image can glow acid yellow, green, cyan, or magenta without changing the image tensor.

images

The image to preview.

fit_mode

Contain + Upscale

Shows the entire image and scales it as large as possible without cropping.

Unused space can remain when the image and node have different aspect ratios.

Cover

Fills the entire preview region while preserving image proportions.

Overflow is cropped from the preview only.

Fit Width

Matches the preview width.

The image may extend beyond the visible height.

Fit Height

Matches the preview height.

The image may extend beyond the visible width.

Stretch

Fills the preview region exactly.

This can distort the preview's proportions but does not alter the real image tensor.

Actual Size

Displays the image at its natural browser pixel dimensions, centered inside the preview region.

`background_mode`

Solid

Uses `background_color`.

Checkerboard

Uses a dark checkerboard pattern.

Useful for transparency-oriented inspection.

Blurred Image

Fills the background with a darkened, blurred cover-scaled copy of the image, then draws the selected fit mode above it.

`background_color`

Hex color used by Solid mode.

Example:



#111111

Output: `images`

The original image passes through unchanged.

Chapter 6: Persist + Display Resolved Prompt

Category: Sick Ollie/Prompt

This optional utility accepts a string, displays it, preserves it in saved PNG workflow metadata, and returns the same string.

Prompt Core already contains its own saved final-prompt display. This separate node remains useful because it is general-purpose.

Use it for:

- another prompt-building system
- regex output
- translated prompts
- postprocessed prompts
- any final string that should remain visible after PNG reload

Controls

text

Forced string input containing the exact resolved prompt or text.

saved_prompt

Read-only display restored from the saved workflow.

The node writes the text into:

- the workflow's widget values
- a named node property
- an independent **resolved_prompt** PNG metadata field

The layered storage is intentional. It makes restoration resilient across different reload paths.

Output: **text**

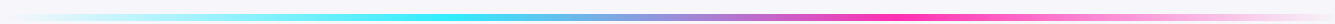
The same string received by the input.

This allows the node to remain inline:

```
Prompt builder
  → Persist + Display Resolved Prompt
  → Generation Core
```

or act as a side branch:

```
Prompt Core final_prompt
  └─ Generation Core positive_text
  └─ Persist + Display Resolved Prompt text
```



Complete Wiring Guide

Essential production wiring

```
Loader Core model
  → Generation Core model

Loader Core clean_name
  → Prompt Core name_value

Loader Core clean_name
  → Output Core clean_name

Loader Core raw_stem
  → Output Core raw_stem

Loader Core diffusion_model_stem
  → Output Core model_name

Loader Core applied_loras
  → Output Core applied_loras

Prompt Core final_prompt
  → Generation Core positive_text

Generation Core samples
  → VAE Decode samples

Generation Core vae
  → VAE Decode vae

Generation Core seed_used
  → Output Core seed

Generation Core generation_info
  → Output Core generation_info

VAE Decode IMAGE
  → Output Core images

Output Core images
  → Preview Core images
```

Optional prompt-log naming wiring

```
Prompt Core prompt_index_resolved  
  → Output Core prompt_index  
  
Prompt Core outfit_index_resolved  
  → Output Core outfit_index  
  
Prompt Core scene_index_resolved  
  → Output Core scene_index  
  
Prompt Core prompt_file  
  → Output Core prompt_file  
  
Prompt Core outfit_file  
  → Output Core outfit_file  
  
Prompt Core scene_file  
  → Output Core scene_file
```

Optional resolved-prompt display

```
Prompt Core final_prompt  
  → Persist + Display Resolved Prompt text
```

Workflow Recipes

Recipe 1: Test every checkpoint for one identity

Folder structure:

```
loras/  
└─ People/  
    └─ Becky/  
        ├── becky_epoch_10.safetensors  
        ├── becky_epoch_20.safetensors  
        ├── becky_epoch_30.safetensors  
        └─ becky_epoch_40.safetensors
```

Loader Core:

```
folder_name      = People/Becky  
include_subfolders = off  
main_enabled     = on  
control_after_generate = increment  
loop_folder      = on  
skip_none_during_cycle = on
```

Queue four prompts.

Result:

```
image 1 → epoch 10  
image 2 → epoch 20  
image 3 → epoch 30  
image 4 → epoch 40
```

Output Core:

```
subfolder_var_1 = clean_name  
filename_var_1  = raw_stem  
filename_var_2  = seed
```

Each image enters the tidy **becky** folder while preserving the exact epoch in its filename.

Recipe 2: One identity, randomized outfits

Loader Core:

```
folder_name      = People/Becky  
control_after_generate = fixed
```

Prompt Core:

```
prompt_source = manual
manual_prompt = A direct-flash fashion photograph of NAME wearing OUTFIT.
outfit_enabled = on
outfit_mode = randomize
outfit_loop = on
```

Connections:

```
Loader Core clean_name
→ Prompt Core name_value
```

Every queued prompt keeps Becky but receives a new outfit line.

Recipe 3: Pair prompt and outfit lines sequentially

Prompt Core:

```
prompt_source = log
prompt_mode = increment
prompt_loop = on

outfit_enabled = on
outfit_mode = increment
outfit_loop = on
```

Both raw indexes advance after each queued prompt.

When the logs have different lengths, each loops independently.

This creates repeating combinations whose overall pattern resets after the least common multiple of the two log counts.

Recipe 4: Persistent style LoRA plus cycling identities

Loader Core:

```
folder_name = People
include_subfolders = on
control_after_generate = increment
loop_folder = on
```

Add a secondary LoRA:

```
Styles/sick_light.safetensors
strength = 0.8
enabled = on
```

The identity changes through the primary folder pool while the style remains applied to every image.

`applied_loras` records both.

Recipe 5: Human folders, forensic filenames

Loader Core name outputs:

```
clean_name = becky
raw_stem   = becky_sickollie_krea2_epoch_12
```

Output Core:

```
output_root      = _SickOllie_Art
subfolder_var_1  = clean_name

filename_var_1    = raw_stem
filename_var_2    = model_name
filename_var_3    = prompt_file_stem
filename_var_4    = seed
```

Result:

```
_SickOllie_Art/
├─ becky/
│   └─ becky_sickollie_krea2_epoch_12_SICK_OLLIE_Krea2_FP8_sick_dolls_66630897578501_00001.png
```

Troubleshooting Guide

The main LoRA dropdown shows unrelated files

Check:

- `folder_name`
- `include_subfolders`

Remember:

- exact folder with subfolders off selects only direct children
- subfolders on includes every descendant
- `[All LoRA folders]` intentionally includes the full library

After changing the filter, the dropdown should rebuild immediately.

The main LoRA changes when I select a folder

This is intentional.

Changing `folder_name` or `include_subfolders` automatically selects the first valid LoRA in the newly filtered pool.

The primary cycle keeps producing `None`

Turn on:

```
skip_none_during_cycle
```

`None` will remain manually selectable but leave the automatic cycle.

The primary LoRA is selected but does not affect the model

Check all three active conditions:

```
main_enabled = on
main_lora    ≠ None
main_strength ≠ 0
```

The `main_active` output confirms the final state.

`clean_name` still contains suffixes

Possible causes:

1. `auto_clean_name` is off
2. the regex does not match the filename
3. the rule is case-sensitive
4. the unwanted text is not at the beginning or end expected by `^` or `$`
5. separators differ from the rule

Example problem:

```
filename: becky_Krea2
rule:    _krea2$
```

Fix:

```
(?i)_krea2$
```

`clean_name` became empty

The cleanup rules removed everything.

Loader Core falls back to `raw_stem` when the final cleaned result is empty, but the rules should still be narrowed.

Avoid broad patterns such as:

```
.*
```

`NAME` remains in the prompt

Check:

- `name_value` is not empty
- `name_token` matches the prompt exactly
- the prompt uses `NAME`, not `Name` or `name`
- Loader Core `clean_name` is connected to Prompt Core `name_value`

Token replacement is literal and case-sensitive.

OUTFIT or SCENE remains in the prompt

Check:

- the corresponding feature is enabled
- a log file is selected
- the selected file contains eligible lines
- the token field matches the prompt exactly
- the resolved count is greater than zero

Randomized log indexes become enormous

This is normal.

Random mode chooses a large raw integer. With looping on, Prompt Core resolves it into the valid line range using modulo.

Use the resolved index outputs for filenames.

Random mode keeps selecting the final log line

Check whether loop is off.

A huge random raw index with loop off clamps to the final line.

Turn looping on for true full-log randomization.

New log files do not appear

Restart ComfyUI and hard-refresh after adding, deleting, or renaming `.txt` files.

Confirm the files are under:

```
ComfyUI/input/SickOllieLogs/
```

and inside the appropriate category folder.

Generation Core does not reproduce the last image

Use:

Use Last Queued Seed

The active seed must become the exact previous `seed_used`.

Also confirm that the model, LoRAs, prompt, dimensions, sampler settings, and software state have not changed.

Output Core creates an unexpected filename

Remember the construction order:

1. literal
2. variable slots from top to bottom
3. delimiter between nonempty pieces
4. automatic ComfyUI counter
5. extension

Check the connected values and the chosen variable names.

Output Core shows an empty prompt, model, or LoRA in metadata

Check these connections:

```
Generation Core generation_info
  → Output Core generation_info

Loader Core applied_loras
  → Output Core applied_loras
```

Output Core can auto-detect the upstream model, but Generation Core's settings and Loader Core's active LoRA list should be connected explicitly.

Civitai does not recognize a resource

The local file hash must match a version known to Civitai.

A resource may fail to match when it is:

- locally merged
- converted
- differently quantized
- modified
- retrained

- absent from Civitai

Renaming alone does not prevent matching because identification uses file contents.

Preview Core says “Waiting for image...”

Confirm:

- `images` is connected
- the upstream path executed
- the node is not bypassed
- an image, rather than a latent, is connected

Preview Core expects `IMAGE`, not `LATENT`.

The saved PNG does not reload the workflow

Check:

```
save_workflow_json = on
```

Also confirm that ComfyUI metadata saving has not been globally disabled.

PNG is the most reliable format for full ComfyUI workflow round-tripping.

Glossary

Active LoRA

A LoRA that is enabled, selected, and has a nonzero strength.

Clean Name

The human-facing primary LoRA name after regex cleanup.

Main LoRA

The primary LoRA controlled by the folder-filtered dropdown and automatic progression.

Raw Stem

The primary LoRA filename without its folder or extension, before cleanup.

Resolved Index

The actual valid log index selected after modulo wrapping or clamping.

Raw Index

The stored prompt, outfit, or scene index before resolution.

Prompt Log

A plain-text file containing one reusable prompt per line.

Token

A literal placeholder such as `NAME`, `OUTFIT`, `SCENE`, or `ITEM`.

Queue-Time Progression

Changing a selector or index after each prompt is serialized into the queue, allowing one queue operation to contain different values.

Applied LoRAs

The newline-separated list of LoRAs that actually affected the model, including strengths.

Filename Prefix

The constructed filename before ComfyUI adds its counter and extension.

Compatibility and Release Scope

This pack was built and tested around ComfyUI's legacy LiteGraph workflow renderer.

Frontend behavior can change after major ComfyUI updates. Keep backups of:

- the last known working custom-node release
- important workflows
- generated PNGs containing workflow metadata

This release includes only the six currently working nodes:

- Loader Core
- Prompt Core
- Generation Core
- Output Core
- Preview Core
- Persist + Display Resolved Prompt

Not included:

- unfinished decorative labels
- section panels
- dividers
- badges
- sticky notes
- experimental guide overlays
- superseded Loader, Prompt, Generation, Output, or Preview variants

Those experiments may return after being stabilized individually.

Credits

Created for the Sick Ollie ComfyUI production workflow.

Loader Core's enhanced secondary-LoRA interface uses rgthree-comfy components and conventions. See [THIRD_PARTY_NOTICES.md](#) for attribution and license information.

License

See the repository's [LICENSE](#) file.